Scaling Writes

什麼是寫入擴展？

寫入擴展（Scaling Writes）處理的是當單一 database 或單一 server 成為瓶頸時，如何應對大量寫入操作的挑戰。隨著應用從每秒幾百次寫入成長到幾百萬次，各個元件都會碰上磁碟 I/O、CPU 和網路頻寬的硬性限制，而這些瓶頸，正是面試官最喜歡深挖的地方。

挑戰在哪裡

很多系統設計題目一開始的規模要求都很平易近人，然後面試官會突然拋出一句：「那這個要怎麼擴展？」你可能對讀取端的工具比較熟悉（例如 read replica、caching 等），但寫入端往往是更大的挑戰。

爆發性、高吞吐量、又充滿競爭的寫入，其實是系統設計中最困難的部分之一。不同的架構選擇可能讓你輕鬆過關，也可能讓問題爆炸。面試官很喜歡探究你設計中的瓶頸，看你如何應對他們下一個產品爆紅的假設情境（至於這個假設現不現實，那是另一個討論）。

解法的架構

寫入擴展不只是把更多硬體砸進去而已，我們可以透過架構上的選擇來提升系統承載寫入的能力。以下四個策略組合起來，可以讓你的系統突破單一、未經優化的 database 或 server 的限制：

1. 垂直擴展（Vertical Scaling）與 Database 選型
2. Sharding 與 Partitioning
3. 使用 Queue 和 Load Shedding 處理爆發流量
4. Batching 與階層式聚合（Hierarchical Aggregation）

我們先從如何在單一 server、單一 database 的架構內把效能壓榨到極限開始，再來談往外擴展。

第一步：垂直擴展與寫入優化

buildmost.org

面對寫入挑戰的第一步，是確認你已經把現有硬體的潛力用盡。這樣做是要向面試官展示你做足了功課，不會在硬體或軟體調校就能解決問題的情況下，過早地增加系統複雜度。

Vertical Scaling（垂直擴展）

先從硬體開始，也就是「垂直擴展」。寫入的瓶頸來自磁碟 I/O、CPU 或網路頻寬。在往下走之前，我們需要確認是真的碰到了這些牆。通常這代表要做一點快速的 back-of-the-envelope 估算，確認實際的寫入吞吐量是多少，以及這個量是否超出硬體能承受的範圍。

很多面試者習慣想到的是 4 到 8 個 core、配一顆機械硬碟的機器。但在很多情況下，雲端供應商和資料中心提供的硬體遠比這強大，在需要重新架構之前你還有很多路可以走。

硬體的天花板可能比你想的高得多！擁有 200 個 CPU core 和 10Gbps 網路介面的系統並不罕見。花點時間熟悉一些關鍵效能數字（Numbers to Know），確保你沒有白白浪費潛力。

你可以提出（大多數面試官也會期待或欣賞）部分挑戰可以靠現代硬體解決。這種說法在 staff 級以上的面試者身上最常見，因為他們對效能曲線的邊界有直覺。

不過，很多面試官在評分標準裡就預設了需要討論非垂直擴展的方案，他們可能一直移動目標直到你被逼著用其他方式解決規模問題。所以提這個觀點是對的，但如果面試官沒有接受，就不要僵持下去。

Database 選型

接下來要考慮的是，你底層的資料存儲是否適合你的寫入模式。大多數應用都混合了讀取和寫入，我們在選擇 data store 時需要同時考量兩種存取方式。不過，當系統以寫入為主時，可以把一些只用來加速讀取的功能拿掉，換取更好的寫入效能。

Cassandra 是個很好的例子。Cassandra 靠著 append-only commit log 架構實現了優異的寫入吞吐量。它不是在原地更新資料（這需要昂貴的磁碟尋址），而是把所有資料循序寫入磁碟。這讓它在普通硬體上就能處理每秒 10,000 次以上的寫入，而傳統 relational database 做同樣的事可能只有每秒 1,000 次。

但代價是：Cassandra 的讀取效能不怎麼好。讀取資料往往需要檢查多個檔案並合併結果，可能比索引設計良好的 relational database 還慢。這是一種典型的取捨：用較低的讀取效能，換取更高且穩定的寫入吞吐量，對寫入密集型系統而言是合理的選擇。

> **讀寫之間的張力**：這是寫入擴展中最根本的矛盾。優化寫入效能往往會犧牲讀取效能，反之亦然。你需要搞清楚你的瓶頸在讀還是在寫，然後針對性地優化。系統的

buildmoat.org

> 不同部分可能需要不同的策略。

其他 database 也做了類似的取捨，只是方式不同：

- **Time-series database**，如 InfluxDB 或 TimescaleDB，專為帶有 timestamp 的高頻率循序寫入而設計（它們也有內建的 delta encoding 來提升儲存效率）
- **Log-structured database**，如 LevelDB，採用 append 而非 in-place update
- **Column store**，如 ClickHouse，可以為分析型工作負載有效率地批次處理寫入

選擇正確的 database 是個大話題，但核心洞察是：如果你的寫入量大到值得為此調整架構，你就可以做出有針對性的選擇，讓效能大幅提升。

除此之外，還有一些可以對任何 database 做的寫入優化：

- **停用昂貴的功能**：在高寫入期間，暫時關掉 foreign key constraint、複雜的 trigger、或全文搜尋 indexing
- **調整 write-ahead logging**：像 PostgreSQL 這樣的 database 可以設定在多個 transaction 後才一起 flush 到磁碟
- **減少 index 數量**：index 越少，寫入越快，但會降低讀取查詢的效率

關鍵在於，大多數通用型 database 被設計成合理地處理混合工作負載，但它們並沒有針對極端情況優化。當你知道系統是寫入密集的，就可以做出有針對性的選擇，讓效能大幅提升。

在面試中，提到具體的 database 選型能展示你理解取捨。不要只停留在「使用效能更好的資料庫」這種籠統描述，而是要清楚解釋原因，為什麼 Cassandra 的 append-only 寫入比 MySQL 的 B-tree 更新更快，或者為什麼你會為 metrics 收集選擇 time-series database。

buildnbat.org

第二步：Sharding 與 Partitioning

好，假設我們已經把單一 server 的潛力用盡，需要往水平擴展走了，接下來怎麼做？

如果一台 server 能處理每秒 1,000 次寫入，那 10 台 server 應該（這個「應該」很重要）能處理每秒 10,000 次。理想狀態下，我們可以把寫入負載分散到多台 server，讓每台只處理可管理的量，輕鬆獲得擴展能力。

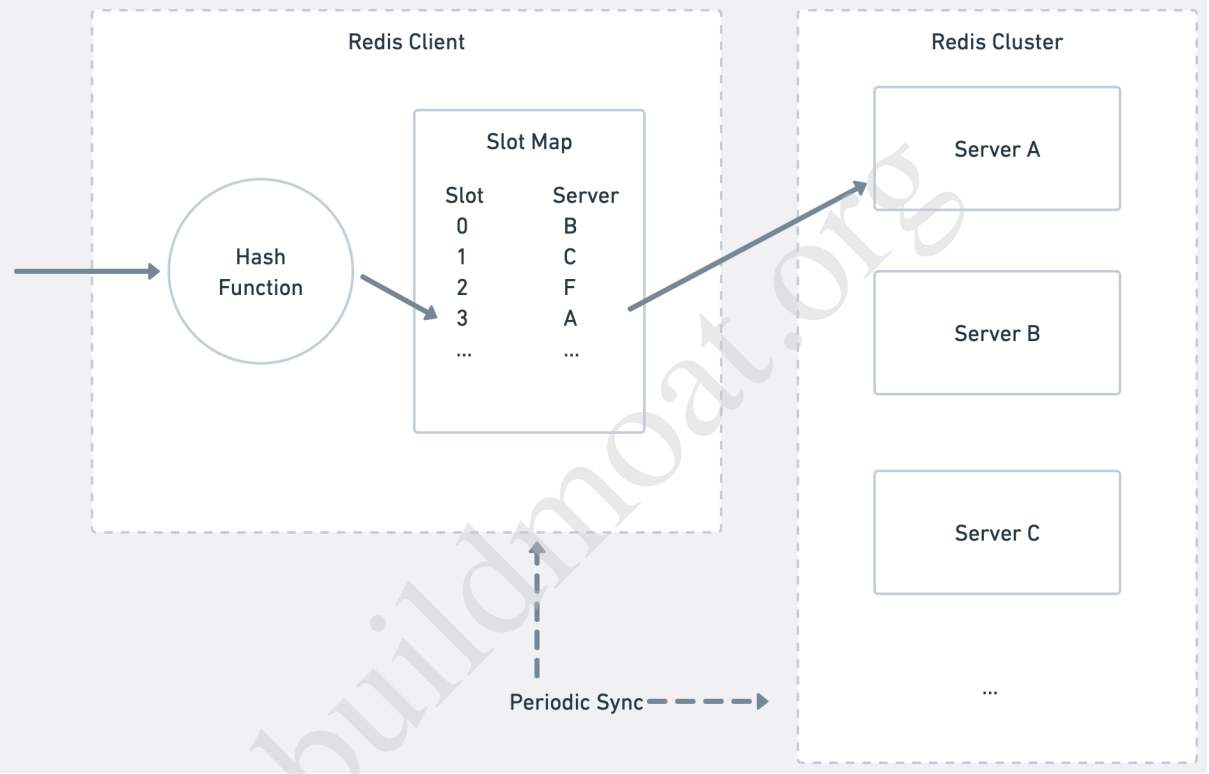
這些額外的 server 通常被稱為「shard」，許多 shard 實際上可以作為一個邏輯 database 存在，我們把它當作一個整體來操作。需要多台 server 這件事，對應用層來說大多是透明的。

Horizontal Sharding（水平分片）

Redis Cluster 是 sharding 的一個簡單好例子。Redis 中每筆資料都用一個字串 key 儲存。這些 key 用一個簡單的 CRC function 做 hash，得到一個「slot number」，然後這些 slot number 被分配給 cluster 裡的不同 node。

Client 會向 Redis Cluster 查詢 cluster 中的 server 及其負責的 slot number。當 client 要寫入一個值時，它 hash 這個 key 得到 slot number，查找負責那個 slot 的 server，然後把寫入請求發給那台 server。

這樣一來，資料就被分散到 cluster 裡所有的 server 上了。另一個常見做法是用 consistent hashing 來決定要寫到哪台 server。



選擇好的 Partitioning Key

實際上，面試官期待你說「sharding」，但他們想知道這具體怎麼運作。你最重要的任務是說清楚如何選擇 partitioning key。如果你選了一個好的 key（比如對 userID 做 hash），資料就會均勻分散在整個 cluster 上，你就實現了假設中透過增加 10 倍 server 獲得 10 倍效能的目標。

很多面試官可以接受「如果有個好的 partitioning key，client 就有辦法找到正確的 server」這個說法。但也不乏面試官會追問這具體是怎麼實現的。熟悉 consistent hashing、virtual node 和 slot assignment 機制，能讓你不被這些問題打個措手不及。

如果選了一個爛的 key 會怎樣？ 假設我們不用 hash userID，而是用用戶的國家作為 key。結果可能是大量寫入集中到人口眾多的中國，而只有很少的寫入落到人口稀少的紐西蘭。紐西蘭的 shard 閒置，中國的 shard 過載。

原則是：我們要選一個讓每個 shard 的寫入量差異最小的 key。從直覺上看，就是「越平衡越好」。通常可以對主要識別碼（像 userId 或 postId）做 hash 來達到這個效果。你應該能從設計中找出幾個明顯的候選識別碼。

同時要記得考慮資料怎麼被讀取。如果你把所有寫入分散到各個 shard，但每個讀取請求都需要從每一個 shard 收集資料，那讀取端的 overhead 會很大，因為每個讀取者都需要向所有 shard 發起網路請求。

對你系統中讀寫最頻繁的資料（有時稱為「最熱的」資料），你應該預期會和面試官討論資料的物理分布方式。你要選一個能盡量均勻分散讀寫、同時把常被一起存取的資料聚在一起的方案。問自己「這個請求需要打到幾個 shard？」以及「這個請求多常發生？」這兩個問題能幫你判斷是否製造了瓶頸。

Vertical Partitioning（垂直分片）

Horizontal sharding 是把 row 分開，而 Vertical partitioning 是把 column 分開。你把有著不同存取模式和擴展需求的不同類型資料分離出來，而不是把所有東西塞進一張巨大的表。

想像一個社群媒體貼文。在單體架構下，你可能有一張表包含所有關於貼文的資料：

```
TABLE posts (
    id BIGINT PRIMARY KEY,
    user_id BIGINT,
    content TEXT,
    media_urls TEXT[],
    created_at TIMESTAMP,
    like_count INTEGER,
    comment_count INTEGER,
    share_count INTEGER,
    view_count INTEGER,
    last_updated TIMESTAMP
);
```

這張表被各種操作同時打：用戶寫入內容、系統不斷更新互動數字、分析 query 掃描大量資料。每種操作都在互相干擾。

用 Vertical partitioning，你可以把它拆成專門的表：

```
-- 貼文核心內容（一次寫入，多次讀取）
TABLE post_content (
    post_id BIGINT PRIMARY KEY,
```

buildmoat.org

```
    user_id BIGINT,
    content TEXT,
    media_urls TEXT[],
    created_at TIMESTAMP
);

-- 互動數字（高頻率寫入）
TABLE post_metrics (
    post_id BIGINT PRIMARY KEY,
    like_count INTEGER DEFAULT 0,
    comment_count INTEGER DEFAULT 0,
    share_count INTEGER DEFAULT 0,
    view_count INTEGER DEFAULT 0,
    last_updated TIMESTAMP
);

-- 分析資料（append-only，time-series）
TABLE post_analytics (
    post_id BIGINT,
    event_type VARCHAR(50),
    timestamp TIMESTAMP,
    user_id BIGINT,
    metadata JSONB
);
```

buildmoat.org

邏輯上拆開之後，我們還可以把每張表移到不同的 database instance，分別針對各自的工作負載優化：

- **貼文內容**：用傳統 B-tree index，針對讀取效能優化
- **互動數字**：可以用 in-memory storage 或專門的計數器，應對高頻率更新
- **分析資料**：使用針對 time-series 優化的儲存，或支援行式壓縮的 database

資料建模的挑戰不只是細節，，更在於你如何從邏輯上規劃資料。這種邏輯設計和資料實際存放的位置同樣重要。

第三步：用 Queue 和 Load Shedding 處理爆發流量

Partitioning 和 sharding 能幫你走完 80% 的路，但在實際生產環境中它們常常碰壁。現實中的寫入流量不是穩定的。雖然大規模系統往往確實會平滑（Amazon 的訂單量出

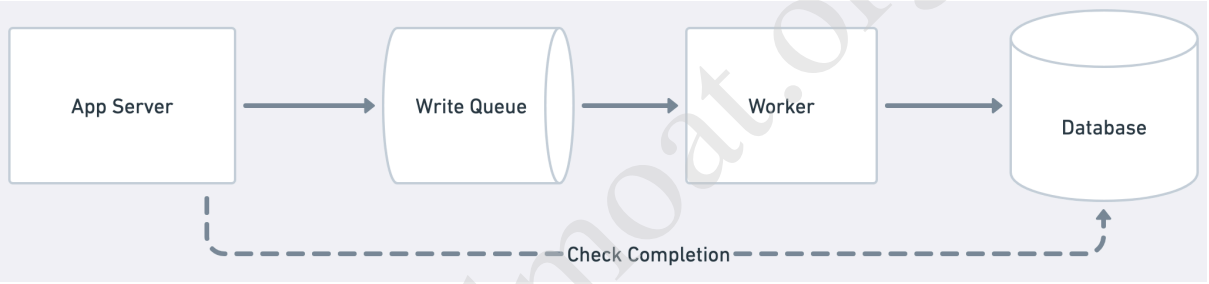
乎意料地穩定），但爆發流量是很常見的。面試官很喜歡問「黑色星期五訂單量變成 4 倍怎麼辦」或「跨年夜路上的司機是平時的三倍怎麼辦」。

如果高峰期需要承受平時 4 倍的寫入量，那意味著平時我們只用了 25% 的總容量。大多數系統根本不是這樣規劃的！很多面試者的第一反應是「用 autoscaling」，autoscaling 確實是個好工具，但它不是萬能藥。擴容縮容需要時間，更糟的是，database 系統在擴展過程中往往需要停機或降低吞吐量。這恰恰是業務最忙的時候，最不想看到的事。

所以我們要麼把寫入先暫存起來，讓系統以最快的速度慢慢處理，要麼以對業務可接受的方式丟棄一些寫入。兩個都來談談。

Write Queue（寫入佇列）：應對爆發流量

大多數面試者第一個想到的是在系統中加一個 queue，比如 Kafka 或 SQS。這把「接受寫入」和「處理寫入」解耦，讓系統能以自己的最大速度處理寫入。



Queue 天生是非同步的，這代表 app server 只知道寫入被記錄到 queue 了，不知道是否已成功寫入 database。大多數情況下，client 需要一種方式回來查詢寫入是否最終完成。對某些場景來說沒問題。

這個方法最重要的好處是**爆發吸收**：queue 作為緩衝，平滑了流量峰值。你的 database 以穩定的速率處理寫入，由 queue 負責吸收爆發。

但 queue 只是暫時的解法。如果 app server 持續以比 database 能消化更快的速度寫入 queue，queue 就會無限制地成長，每筆寫入需要等待的時間越來越長。

雖然我們也許能擴展 database 來處理增加的負載（以及堆積在 queue 裡尚未處理的請求），但這往往反而讓問題更嚴重，因為用戶等得越來越不耐煩。

> Queue 是很強大的工具，但面試者經常沒有考慮到它掩蓋底層問題的情況。**在你預期爆發是短暫的時候使用 queue，不要用它來 patch 一個連穩定負載都扛不住的 database。**

在需求確認階段，就要搞清楚我們對延遲寫入或不一致讀取的容忍度。很多系統可以接受一點點延遲，特別是在流量最高的罕見情況下。如果是這樣，用 queue 可能是個

squidmoat.org

好選擇。但要小心引入 queue 後是否違反了關鍵的功能或非功能性需求。

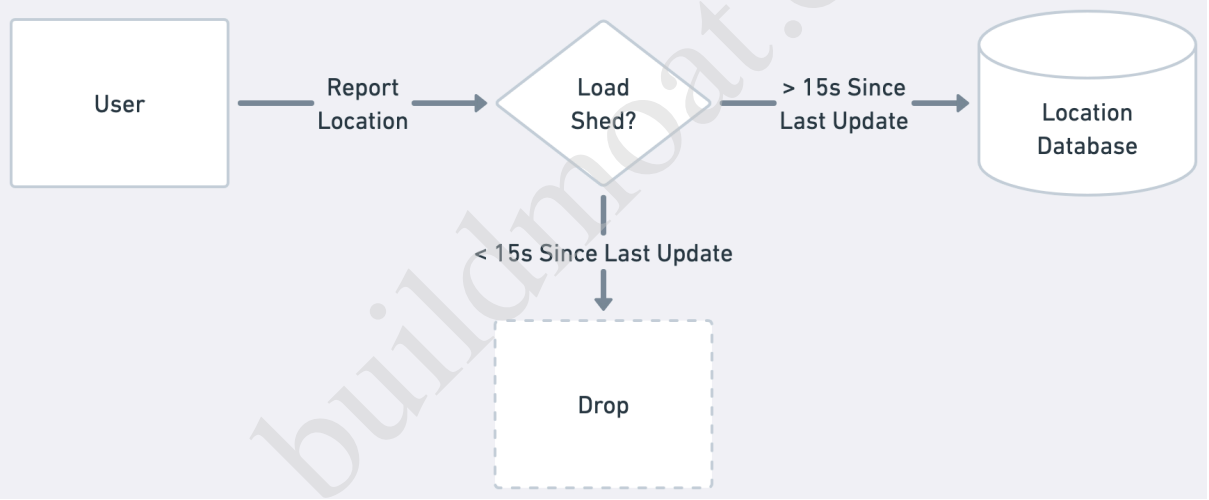
Load Shedding（負載丟棄）

另一個應對爆發的選項看起來像是在逃避，但其實是很強大的工具。當系統不堪重負時，你需要決定哪些寫入要接受、哪些要拒絕。這叫做 load shedding，比讓所有東西一起失敗要好得多。

Load shedding 嘗試判斷哪些寫入對業務最重要、哪些不是。如果我們能丟掉不那麼重要的寫入，就能讓系統繼續運作，更重要的寫入仍然會被處理。

想想 Strava 或 Robotaxi 這類用戶定期回報位置的系統。如果用戶數過多，加 queue 只會讓 queue 爆掉。但退一步想想：用戶每隔幾秒就會再打一次 API 更新位置。如果我們丟掉一次寫入，幾秒後就會收到比那筆更新鮮的資料！

一個簡單的解法是，在系統過載時直接丟棄最不有用的寫入。對 Robotaxi 來說，這可能是距離上一次更新只有幾秒的位置資訊。對分析系統來說，我們可能暫時丟棄 impression 資料，確保更重要的 click 資料能被處理。



根據系統的性質，設置一些「洩壓閥」能展示你有辦法讓糟糕的情況（負載過高）不演變成災難（系統崩潰），即使這意味著部分用戶的體驗暫時不好。

第四步：Batching 與階層式聚合

前面的解法都把現有的寫入視為既定事實，但我們其實常常可以改變寫入的結構，讓它們更容易被處理。單筆寫入操作都有它的 overhead，包括網路來回、transaction 建立、index 更新。此外，大多數 database 處理批次寫入的效率遠高於逐筆寫入。當 database 成為瓶頸時，我們可以往上游看，思考如何讓進來的資料更容易被處理。

Batching（批次寫入）

其中一個做法是把寫入合批。與其一筆一筆處理，不如把多筆寫入合在一起，平攤那些 overhead。這可以在應用層、中間處理層，或甚至在 database 層完成。

應用層（Application Layer）

在應用層，client 可以把寫入先收集起來再一次送到 database。這在應用本身不是資料唯一來源（source of truth）的情況下特別好用。

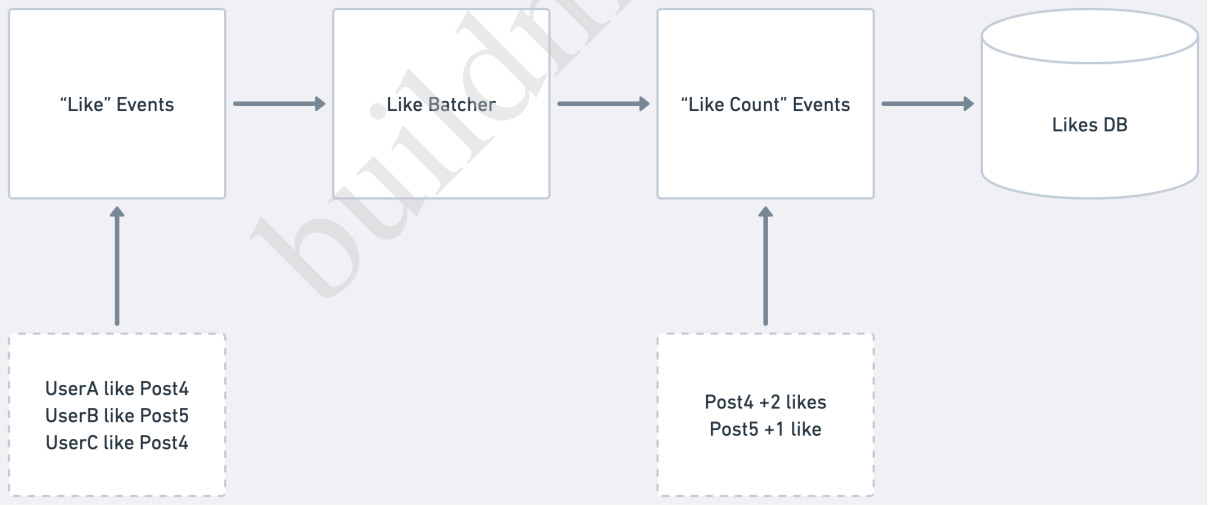
比如有個 service 從 Kafka topic 讀取訊息、做一些處理、然後寫入 database，我們可以把寫入先攢一批再送出去。如果應用崩潰，重新讀取 Kafka topic 就能復原，資料不會遺失。

但如果應用本身是資料的 source of truth，就需要能承受潛在的資料遺失風險。最壞的情況是：用戶的請求被確認、放進一個 batch，然後服務在 batch 被提交到 database 之前崩潰了。不是所有應用都能接受這種情況。

中間處理層（Intermediate Processing）

另一個選項是加一個中間處理程序，在寫入到 database 之前先把資料合批。想像一個接收「按讚」事件的系統，需要追蹤每篇貼文的按讚數。

我們的 Like Batcher 可以讀取一批這樣的事件、統計每篇貼文的按讚數變化，然後把這些變化一次送到 database。如果一篇貼文在一個時間窗口內（比如一分鐘）收到 100 個讚，我們就把 100 次寫入降為 1 次！



> **Staff 級面試者需要更深入**：如果大多數貼文每小時只得到一個讚，把批次頻率設為一分鐘根本沒有任何幫助！我們需要確認 batching 本身真的帶來了收益。

Database 層（Database Layer）

最後一層是 database 本身。大多數 database 系統都有可以調整的選項，控制多久把寫入 flush 到磁碟一次，而磁碟通常是系統的主要瓶頸。

buildmoat.org

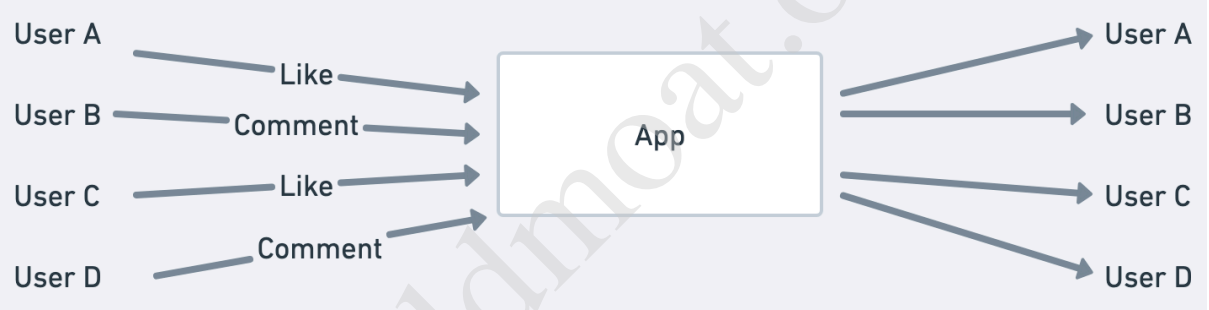
以 Redis 為例，預設配置是每 100ms flush 一次寫入到磁碟。這意味著如果一個 batch 有 1000 筆寫入，我們只在最後一筆寫入後的 100ms 才真正寫磁碟一次。

在 database 層做這件事有一定的優雅感，但這絕對是「大鎚」解法，應該留給極端情況。

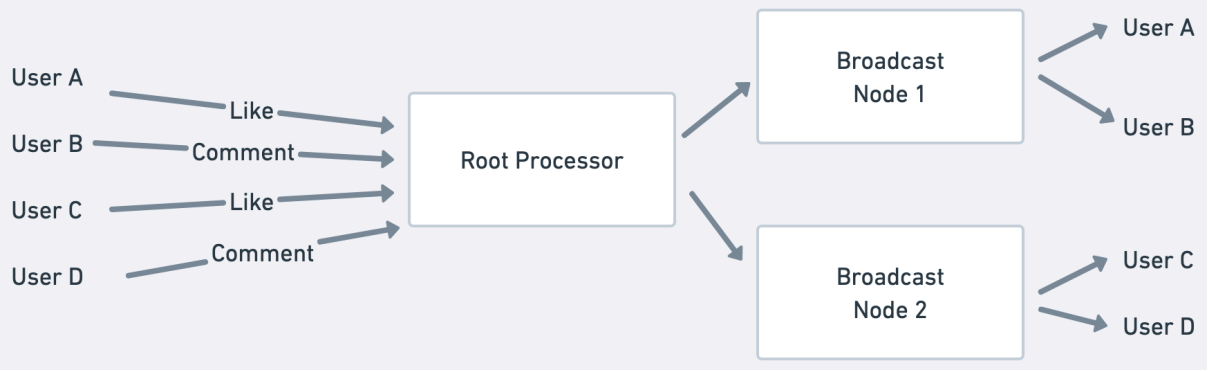
Hierarchical Aggregation（階層式聚合）

這個策略適用於最極端的情況。對分析和串流處理這類高量資料，你往往不需要儲存每一個獨立的事件，而是需要聚合後的視圖。關鍵洞察是：這些視圖可以一步一步遞增地建立出來。Hierarchical aggregation 以多個階段處理資料，在每個步驟減少資料量。

我們來看一個具體的例子。在直播影片串流中，觀眾可以留言，也可以對留言按讚。每當一個觀眾做了其中一件事，所有其他用戶都需要收到通知。如果有幾百萬個觀看者，這就製造了一個棘手的局面：幾百萬人在寫入，而所有這些寫入都需要傳送給其他所有人。



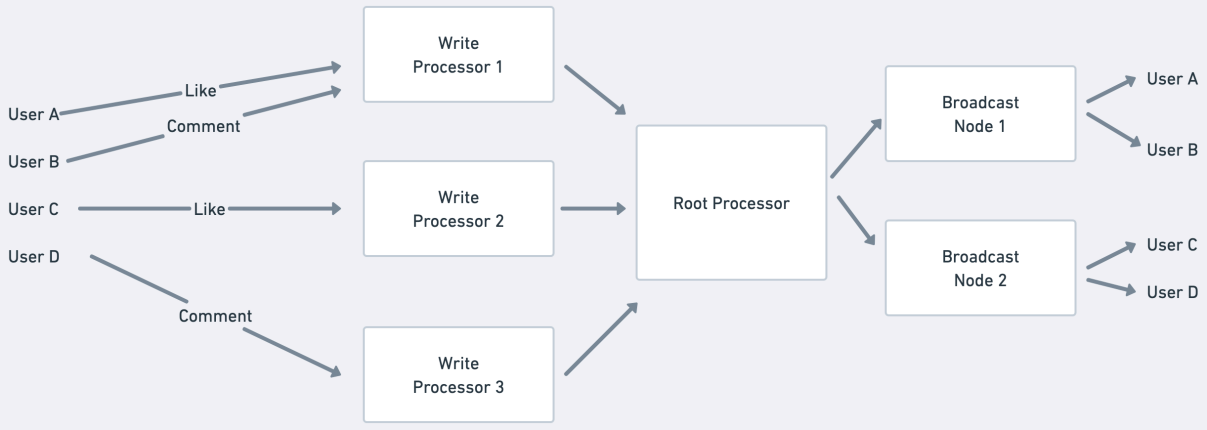
但等等，我們可以簡化一下。所有觀看者想要的是同一個、最終一致的視圖：他們想看到所有最新的留言和對應的數字。我們可以用 consistent hashing 把用戶分配到 broadcast node 上。與其讓每筆寫入分別傳給每個人，我們改成寫出到 broadcast node，由它們再轉發更新給各自負責的觀看者。



這樣一來，原本要寫給 N 個觀看者，現在只需要寫給 M 個 broadcast node。

buildmoat.org

但還有一個問題：我們的 root processor 接收來自所有觀看者的事件。同樣地，我們可以用類似的方式處理：



根據留言的 ID（或對於按讚，是被按讚的留言 ID）來選擇要呼叫哪個 write processor。Write processor 可以對它們負責的留言，在一個時間窗口內聚合按讚數，然後把一批更新轉發給 root processor，root processor 只需要做合併就好。

透過 write processor 聚合資料、再由 broadcast node 分發資料，我們大幅減少了任何單一系統需要處理的寫入量，但也因此增加了部分延遲。。這就是 hierarchical aggregation 的核心！

什麼時候在面試裡用這些

寫入擴展的模式在很多高規模的系統設計面試中都會出現。你不要等面試官來問，而是主動識別瓶頸、驗證它們，並提出解法作為 deep dive 的主題。

一個厲害面試者的回應範例：

> 「有幾百萬用戶在發內容，我們很快就會碰到寫入瓶頸。讓我先估算一下我們的寫入吞吐量……好，這個量確實很可觀！我會在 deep dive 裡回來處理這個問題。」

> 「對於發文的寫入，我認為用 user ID 做 database partitioning 是合理的，這樣可以讓負載均勻分散到各個 shard 上。我們還是需要處理單一用戶大量發文的情況，但可以搭配 queue 和 rate limiting 來解決。」

常見面試情境

Instagram / 社群媒體：適合展示用 user ID 做 sharding，對不同資料類型（用戶 profile、貼文、分析資料）做 vertical partitioning，以及對舊貼文使用階層式儲存。

News Feed：需要在兩個方向之間細心調校，像是名人貼文的寫入量（需要寫入幾百萬個追蹤者的 feed）和這幾百萬用戶來看 feed 時的讀取量之間的平衡。

buildmoat.org

搜尋應用（Search Applications）：搜尋應用往往是寫入密集的，因為在寫入階段需要大量的前處理，才能讓搜尋結果快速回傳。Partitioning 和 batching 是讓這件事成立的關鍵。

直播留言（Live Comments）：直播留言是 hierarchical aggregation 的絕佳案例，可以避免幾百萬個觀看者需要各自和其他所有人通訊這種無法解決的 all-to-all 問題。

什麼時候不適合用

根本不需要擴展的時候，千萬不要硬用寫入擴展策略！如果你看到某個地方看起來像瓶頸，先做一個快速的 back-of-the-envelope 估算，確認值不值得花這個力氣。

每種策略都有取捨。Queue 意味著最終一致性和延遲，partitioning 可能讓讀取路徑變複雜，batching 增加延遲和複雜的活動元件。在提出方案之前，先向面試官展示你清楚這些取捨。最糟糕的情況是製造出一個原本不存在的問題！

常見的 Deep Dive 問題

以下是面試中最常遇到的追問。

「需要增加 shard 數量時，resharding 怎麼處理？」

這是 sharding 最經典的運維挑戰。你一開始有 8 個 shard，現在需要 16 個。如何在不停機的情況下遷移資料？

最天真的做法是讓系統下線、重新 hash 所有資料、把它們移到新的 shard。但對大型資料集來說，這意味著好幾個小時的停機。

生產系統使用**漸進式遷移（gradual migration）**，同時把寫入打向兩個地方（舊的 shard 和新的 shard）。這讓我們可以在保持可用性的同時漸進地遷移資料。

雙寫（dual-write） 階段確保在遷移過程中沒有資料遺失。你同時寫入舊 shard 和新 shard，但讀取時優先讀新 shard。這樣可以在保持可用性的同時漸進遷移資料。

「某個 hot key 的流量多到連一個 shard 都扛不住怎麼辦？」

我們前面談過要讓負載均勻分散在各個 shard 上。但有時候，即使選了最好的 key，某個 shard 還是會收到不成比例的流量。想像一則爆紅的推文每秒收到 100,000 個按讚，就算我們把所有推文均勻分散在 shard 上，這一則推文還是可能把它所在的 shard 搞垮。就算把整個 shard 只用來服務這一則推文，也不夠。

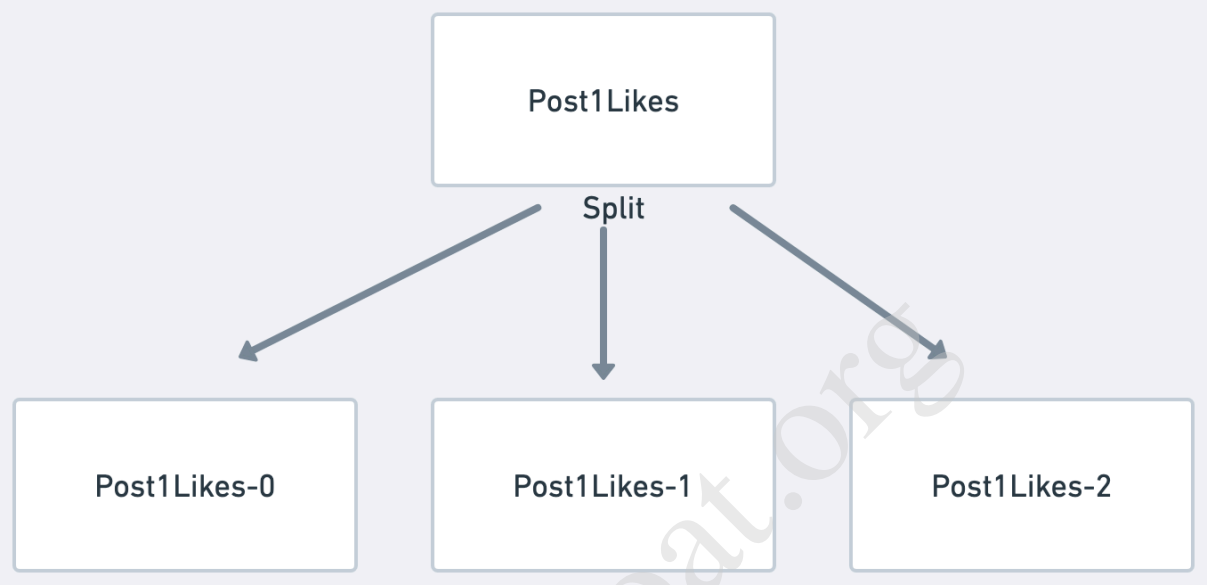
我們有兩個主要選項：

方法一：固定地把所有 key 拆分（Split All Keys）

buildmoat.org

把每個 key 固定拆分成 k 份。這是個大鎚解法，但最簡單。與其把一篇貼文的按讚數放在一個 shard 上，改成分散到多個 shard 上。

對 `post1Likes` 這個 key，拆成 `post1Likes-0`、`post1Likes-1`、`post1Likes-2`，一直到 `post1Likes-k-1`。這樣每個 shard 只持有某個貼文的部分資料，而每個 shard 對於單一貼文的寫入量降低了 k 倍。



這有幾個明顯的缺點：

- 整體資料集大小增加了 k 倍
- 讀取量也乘上了 k 倍——要得到一篇貼文的按讚數，需要分別讀 `postId-0`、`postId-1`……一直到 `postId-k-1`

但如果一個較小的 k 值就能讓你的工作負載舒適地回到 database 的寫入容量範圍內，問題就解決了。

方法二：動態拆分 Hot Key（Split Hot Keys Dynamically）

另一個解法是根據一個 key 是否「熱」，動態地把它拆成多個 sub-key。

對爆紅推文的例子，你可能把按讚數分散到 100 個 sub-key，每個處理每秒 1,000 個按讚。讀取時，從所有 sub-key 聚合計數。

這兩種方法都適用於可以被聚合的 metrics（按讚數、觀看數、計數、餘額），但不適用於必須保持原子性的資料（用戶 profile）。幸運的是，後者幾乎不會承受前者那樣的寫入壓力。

重要的是，**讀者和寫者都需要知道哪些 key 是熱的**，這樣整個方案才能運作。我們有兩個主要做法：

buildmoat.org

- **讀者永遠檢查所有 sub-key**：跟 key split 方法一樣的讀取放大，但實作簡單易懂。寫者在偵測到一個 key 可能是熱的時候（可以維護本地統計數據），條件性地寫入 sub-key。
- **寫者向讀者廣播 split**：所有讀者都需要在 split 執行前收到這個通知。實作更複雜，但更有效率，也能避免讀者去讀不存在的 sub-key。

大多數生產系統使用第一種方法，因為它更簡單，而且檢查 sub-key 的 overhead 相比正確處理 hot key 的效能收益來說微不足道。面試中不要過度工程化你的解法！

總結

寫入擴展歸結為四個協同運作的基本策略：垂直擴展與 database 選型、sharding 與 partitioning、queue 與 load shedding、batching 與多步驟聚合器。最成功的面試不是把這些概念搞得多複雜，而是在正確的地方識別到需要它們，然後策略性地應用。

一個很容易犯的錯誤是，在根本不需要擴展的地方也硬套寫入擴展策略！如果看到某個地方看起來像瓶頸，先做一個快速的 back-of-the-envelope 估算，確認值不值得花這個力氣。

Sharding 和 partitioning 是嘗試擴展系統的好起點。這是一個簡單的策略，能帶來可觀的效益，而且大多數面試官都會期待看到它。

如果你在處理**高量的分析或數值資料**，batching 和 hierarchical aggregation 可以立刻帶來 5 到 10 倍的改善。

最後，**queue 和 load shedding** 在需求允許非同步處理或甚至允許丟棄請求的情況下是很棒的工具。在梳理需求的時候，留意看看它們是否適合這個場景。

核心洞察是：寫入擴展就是降低每個元件的吞吐量壓力。不管是把 10,000 次寫入分散到 10 個 shard、用 queue 平滑爆發、還是把寫入批次成 100 次批量操作，這都是你在應用同一個原則：讓每個單一元件只需要處理可管理的負載。

buildmoat.org